INT4418 — DỮ LIỆU LỚN

Bài tập lớn — Nhóm 8: AI Math Tutor

BÁO CÁO BÀI TẬP LỚN

THEO DÕI TRẠNG THÁI NGƯỜI HỌC THỜI GIAN THỰC VÀ GỢI Ý TOÁN CÓ KIỂM CHỨNG TRÊN LUỒNG SỰ KIỆN

Phân tích luồng học tập và phản hồi toán có kiểm chứng

TV1: Lê Thiện Văn – B25CHHT120

TV2: Đinh Hải Dương – B25CHHT090

TV3: Nguyễn Quốc Việt – B25CHHT121

TV4: Đặng Thuỳ Trang – B25CHHT116

TV5: Lê Thiện Đức – B25CHHT085

TV6: Phạm Minh Hiếu – B25CHHT095

TV7: Vũ Hải Nam – B25CHHT107

Số liệu sinh từ results/raw tại thời điểm dựng tài liệu; ma trận thí nghiệm: 115/115 run hợp lệ.

Năm 2026

Mục lục

Mục lục 2
Tóm tắt 4
1. Giới thiệu 5
2. Công trình liên quan 5
3. Dữ liệu 6
3.1. Kho bài toán 6
3.2. Sự kiện mô phỏng 7
4. Phương pháp và kiến trúc 8
4.1. Đưa sự kiện vào Kafka 8
4.2. Trạng thái theo người học trên Spark 9
4.3. Ghi kết quả idempotent và đối soát 9
4.4. Dịch vụ gợi ý có kiểm chứng 9
4.5. Batch baseline 10
4.6. Độ phức tạp 10
4.7. Tổng hợp theo (người học, bài) bằng MapReduce 10
5. Thiết lập thí nghiệm 11
Phân tích thống kê 12
6. Kết quả 13
6.1. RQ1 — Độ tươi và chi phí: streaming so với batch 13
6.2. Mở rộng theo tải, partition và số người học 14
6.3. RQ3 — Watermark, sự kiện muộn và công bằng 15
6.4. RQ2 — Gợi ý đúng, có ích và đủ nhanh 16
6.5. Chạy dài và trần ingestion 17
6.6. Thí nghiệm mở rộng MapReduce 17
6.6.1. Phân tích lỗi 20
7. Ablation, phân tích lỗi và các mối đe dọa tính hợp lệ 20
7.1. Ablation 20
7.2. Phân tích lỗi 21
7.3. Các mối đe dọa tính hợp lệ 21
7.4. Quyền riêng tư và đạo đức 23
8. Kết luận và hướng phát triển 23

Tài liệu tham khảo 25
Phụ lục A. Khai báo sử dụng AI 27
Phụ lục B. Phân công nhiệm vụ 27
Phụ lục C. Trường của một run record 27
Phụ lục D. Lệnh tái lập 28
Phụ lục E. Bảng số liệu đầy đủ 28

Tóm tắt

Hệ thống gia sư cần biết người học đang bế tắc ngay khi việc đó xảy ra, nhưng sự kiện học tập đến liên tục, có bản trùng, bản muộn và đảo thứ tự. Nhóm xây dựng một pipeline Kafka + Spark Structured Streaming giữ trạng thái từng người học theo event time (khử trùng, watermark, transformWithState trên RocksDB), ghi Redis một cách idempotent, và một dịch vụ gợi ý ba bậc tách khỏi đường luồng, trong đó mọi biểu thức đều là bước giải đã được SymPy kiểm. Dữ liệu là sự kiện mô phỏng có ground truth trên kho 1.000 bài toán lấy từ NuminaMath-CoT. Ma trận thí nghiệm gồm 115 run, mỗi cấu hình 3 seed (trừ hai mục stress), báo trung vị và min–max qua seed.

Ở tải mặc định 1.000 sự kiện/giây, P95 độ trễ cập nhật trạng thái là 4,25 s (4,07–4,54) (SLO 5 s). Thời gian phát hiện bế tắc (trung vị) của batch theo ngày là 62.947 s (61.267–80.977), của streaming là 3,84 s (3,81–4,03), tỉ số 16.388× (15.195–21.237). Với hồ sơ trễ cao và W = 2 phút, trạng thái khớp oracle 80,8% (80,2–80,8) ở chế độ áp ngay và 100,0% (100,0–100,0) ở chế độ đệm theo watermark; với W = 30 giây, người học dùng mạng di động bị loại 8,64% (8,58–8,70) sự kiện so với 0,85% (0,82–0,86) ở wifi. Audit 120 gợi ý do hai thành viên nhóm chấm mù (rà và sửa phiếu điền sẵn nhãn nháp) cho hint validity 79/120 = 65,8% [57,0–73,7%] (κ = 0,781 (bootstrap 95%: 0,643–0,898)); bộ sinh theo bài đạt 57/60 = 95,0% [86,3–98,3%], bộ mẫu chung 22/60 = 36,7% [25,6–49,3%]. API gợi ý có P95 4,9 ms (4,9–5,4) ở 100 người dùng đồng thời (SLO 300 ms).

Ở cùng 1.000 sự kiện/giây, P95 độ trễ hệ thống 1,78 s (1,76–1,80) ở 100, 4,25 s (4,07–4,54) ở 1.000, 38,08 s (37,92–38,40) ở 10.000 người học; ở 10.000 người học 0/3 run đạt SLO 5 s. Soak 30 phút: P95 độ trễ hệ thống 5,04 s (n = 1) (đạt SLO cả run: 0/1 run), 54,7% (n = 1) số cửa sổ một phút có P95 vượt 5 s, RSS driver tăng 108 MB/phút (n = 1) (5 phút cuối / 5 phút đầu: 1,73× (n = 1)). Nhận định 'đạt SLO 5 s ở 1.000 ev/s với P = 3 hoặc 6' chỉ đúng cho các run 10 phút: chạy 30 phút (soak) thì cấu hình mặc định không giữ được SLO. Trần ingestion của bộ phát lại và Kafka một nút: 6/6 ô giữ được tốc độ mục tiêu; cao nhất đạt 60.255 ev/s (mục tiêu 60.000 ev/s, 4 tiến trình gửi), nên trần nằm trên mức cao nhất đã thử (60.000 ev/s). Ablation: khớp oracle 93,1% (93,1–93,9) ở cấu hình mặc định, 21,1% (20,9–21,2) khi tắt khử trùng, 100,0% (100,0–100,0) với ordered (P95 136,85 s (136,53–137,10)) và 93,2% (93,1–93,8) với BKT. Một job MapReduce tổng hợp sự kiện theo (người học, bài) chạy trên ma trận 3 quy mô × 2 cấu hình tài nguyên × 3 lần lặp: ở 10.000 người học (1.256.768 lần giao), job xong trong 15,10 s (15,04–15,16) với Config A và 8,45 s (8,40–8,58) với Config B (A/B = 1,79× (1,75–1,80)), throughput B 148.730 (146.476–149.633) lần giao/s; 18/18 run khớp chính xác bản tham chiếu tuần tự và số đếm của bộ mô phỏng.

1. Giới thiệu

Hệ thống gia sư thông minh cần biết người học đang ở đâu ngay lúc này: ai đang bế tắc, kỹ năng nào chưa vững, gợi ý nào phù hợp. Cập nhật hồ sơ theo lô mỗi ngày thì đơn giản nhưng bỏ lỡ thời điểm can thiệp. Chuyển sang xử lý luồng lại phát sinh vấn đề khác: sự kiện đến muộn hoặc bị gửi lặp khi mạng chập chờn, trạng thái phụ thuộc thứ tự sự kiện (chuỗi đúng liên tiếp, knowledge tracing), và độ trễ đuôi phải giữ thấp khi tải tăng.

Vai trò của dữ liệu lớn trong đề tài nằm ở tốc độ đến và độ đúng, không ở dung lượng. Một người học hoạt động phát vài sự kiện mỗi phút, nên vài nghìn sự kiện mỗi giây tương ứng hàng trăm nghìn người học đồng thời, trong khi trạng thái của mỗi người chỉ vài chục kỹ năng. Khó khăn thật là giữ trạng thái đúng khi dữ liệu không đúng thứ tự, chịu lỗi khi tiến trình chết giữa chừng, và đo độ trễ đuôi một cách trung thực.

Báo cáo trả lời ba câu hỏi:

- **RQ1.** Xử lý luồng rút ngắn thời gian phát hiện người học bế tắc bao nhiêu so với batch theo ngày và theo giờ, và tốn thêm bao nhiêu tài nguyên?
- **RQ2.** Đánh giá đồng thời độ đúng toán học, giá trị sư phạm và độ trễ của gợi ý như thế nào?
- **RQ3.** Ngưỡng watermark và khử trùng ảnh hưởng thế nào tới độ đúng trạng thái khi có sự kiện muộn và trùng, và người học dùng mạng di động có bị thiệt hơn không?

Đóng góp của nhóm:

- **Pipeline trạng thái chịu lỗi:** Kafka key theo người học, Spark transformWithState ba chế độ (áp ngay, đệm theo watermark, không watermark), sink Redis idempotent, đối soát với oracle sau mỗi run.
- **Dịch vụ gợi ý có kiểm chứng:** thang ba bậc không dùng mô hình ngôn ngữ lúc chạy, mọi biểu thức là bước đã kiểm SymPy, thiếu trạng thái thì lùi về bậc an toàn nhất.
- **Bộ đo tái lập được:** bộ mô phỏng có ground truth, bộ phát lại open-loop, runner ghi một run record mỗi lần chạy (Phụ lục B của hướng dẫn) cho ma trận 115 run, và giao thức thống kê đăng ký trước khi có dữ liệu.
- **Đánh giá gợi ý theo rubric:** audit phân tầng gợi ý của tập test, hai thành viên nhóm rà và sửa phiếu điền sẵn, mù nguồn sinh, báo hint validity với khoảng Wilson và Cohen's κ.

2. Công trình liên quan

Mô hình hóa người học. Knowledge tracing của Corbett và Anderson [1] coi mỗi kỹ năng là một trạng thái ẩn hai giá trị, cập nhật sau mỗi lần làm bài. Deep Knowledge Tracing [2] thay mô hình này bằng mạng hồi quy. Cả hai cập nhật tuần tự theo từng

lần làm, nên thứ tự sự kiện ảnh hưởng tới kết quả; đây là điểm nối trực tiếp với vấn đề sự kiện đến muộn trong xử lý luồng.

Bế tắc (wheel spinning). Beck và Gong [3] định nghĩa wheel spinning là tình trạng người học không thành thạo một kỹ năng sau nhiều cơ hội luyện tập (mốc 10 cơ hội). Matsuda và cộng sự [4] kết hợp knowledge tracing với mạng nơ-ron để phát hiện bế tắc, đạt recall 0,79 và precision 0,25 trên dữ liệu Geometry Cognitive Tutor. Các công trình này đo độ chính xác phát hiện trên log có sẵn, không đo phát hiện sớm được bao lâu khi dữ liệu đến dạng luồng.

Gợi ý tự động. Pardos và Bhandari [5] cho thấy gợi ý toán do ChatGPT sinh mang lại mức tiến bộ tương đương gợi ý do người biên soạn. Văn bản do mô hình ngôn ngữ sinh không tự mang bảo đảm đúng toán học, nên nhóm kiểm mọi biểu thức trong gợi ý bằng SymPy [9]. Scarlatos và cộng sự [14] đề xuất rubric năm nhãn nhị phân cho phản hồi tự động (đúng, không lộ đáp án, có hướng làm, chẩn đoán lỗi, giọng tích cực); nhóm dùng rubric này cho audit, kèm khoảng Wilson [16] và Cohen's κ diễn giải theo quy ước Landis–Koch [15].

Xử lý luồng có trạng thái. Mô hình Dataflow [7] đặt nền cho xử lý dữ liệu không có điểm kết thúc và đến sai thứ tự thông qua watermark và trigger, và nêu rõ đánh đổi giữa độ đúng, độ trễ và chi phí. Structured Streaming [6] hiện thực các ý tưởng này trên Spark với API khai báo và exactly-once khi nguồn phát lại được và sink idempotent. Spark 4 bổ sung transformWithState (biến trạng thái, timer, TTL) [12]; dropDuplicatesWithinWatermark khử trùng trong ngưỡng watermark [13]; Kafka [11] là log sự kiện bền có partition theo khóa. Karimov và cộng sự [8] chỉ ra rằng cách đo (tách bộ sinh tải khỏi hệ thống, độ trễ theo thời gian sự kiện, throughput bền vững) quyết định độ tin cậy của benchmark luồng.

Khoảng trống. Trong phạm vi các nguồn đã khảo sát, công trình mô hình hóa người học thường đánh giá trên log offline, còn công trình xử lý luồng đánh giá bằng workload chung. Nhóm chưa thấy đánh giá nào đồng thời xét độ đúng trạng thái người học dưới sự kiện muộn và trùng, thời gian phát hiện bế tắc theo độ tươi của pipeline, và thiên lệch theo chất lượng mạng. Đề tài nhắm vào khoảng này ở quy mô môn học.

3. Dữ liệu

Đề tài không dùng dữ liệu người thật. Có hai nguồn: kho bài toán có lời giải từng bước và bộ mô phỏng sự kiện học tập có ground truth. Chi tiết nguồn, giấy phép và hạn chế nằm trong data card của repo.

3.1. Kho bài toán

Bài toán lấy từ NuminaMath-CoT (giấy phép Apache-2.0, revision ghim), chỉ giữ bài có đáp án là số hữu tỉ hoặc tập nghiệm nhỏ và phương trình một ẩn bậc không quá hai. Kho bài v2 có 1.260 bài: tập test 1.000 bài (720 bài số học lời văn, 280 phương trình

bậc không quá hai) và tập dev 260 bài. Lời giải từng bước của tập test được tách phép tính từ lời giải gốc (706 bài), sinh bằng SymPy (280 bài) hoặc viết trực tiếp (14 bài); mọi bước đều qua bộ kiểm tự động: phép tính đúng chính xác trên số hữu tỉ, phương trình giữ nguyên tập nghiệm, bước cuối bằng đáp án.

Kiểm máy không bắt được đề sai hoặc lời giải khó theo dõi, nên hai thành viên nhóm rà soát độc lập 100 bài test lấy ngẫu nhiên: đạt chung 96/100 = 96,0% [90,2–98,4%], 2 bất đồng, Cohen's $\kappa = 0{,}658$, đạt ngưỡng đăng ký trước. Hai thành viên nhóm không chấm từ phiếu trống: theo yêu cầu của nhóm, mỗi người nhận phiếu Excel đã điền sẵn nhãn của một lượt chấm thử bằng mô hình (hai lượt độc lập), người chấm 1 nhận lượt 1, người chấm 2 nhận lượt 2; mỗi người đọc từng dòng và sửa nhãn mình không đồng ý, làm độc lập. So với bản điền sẵn, người chấm 1 sửa 0 / 3 / 56 ô nhãn, người chấm 2 sửa 2 / 5 / 25 ô (rà soát kho bài / vòng hiệu chỉnh / audit chính). Kết quả vì vậy là nhãn do người rà và sửa, không phải nhãn chấm từ đầu; κ có thể bị đẩy cao vì cả hai người bắt đầu từ bản nháp (neo theo bản nháp), dù hai bản nháp là hai lượt độc lập.

Sau khi đo xong, mỗi bài được thêm đề tiếng Việt (statement_vi), đề gốc tiếng Anh vẫn giữ. Bản dịch phải có đúng tập số của đề gốc, vì số xuất hiện trong đề không bị bộ kiểm gợi ý coi là lộ đáp án, và giữ nguyên mọi đoạn LaTeX của bài phương trình. Bước giải, đáp án và chỉ mục bài tương tự không đổi, nên mọi số đo trong báo cáo vẫn áp dụng.

Trước đó, một vòng chấm thử bằng mô hình trên kho v1 (không phải kết quả chấm của người) đã chỉ ra các nhóm lỗi trong cách sinh lời giải; kho v2 sửa các nhóm lỗi này, và các sửa đổi được kiểm bằng bộ kiểm máy và test.

3.2. Sự kiện mô phỏng

Bộ mô phỏng sinh người học có trạng thái ẩn BKT theo kỹ năng [1] và ba nhóm năng lực, làm các bài của tập test; bước sai được lấy từ các lỗi thường gặp đã kiểm bằng SymPy. Mỗi sự kiện (mở bài, nộp bước, sai đáp án, xin gợi ý, hoàn thành) tuân theo hợp đồng LearnerEvent, có event_time (lúc thao tác), sent_time (lúc gửi lần đầu) và số thứ tự trong phiên. Lớp mạng gán mỗi người học một loại kết nối (wifi hoặc cellular), chèn độ trễ, sự kiện muộn (gửi sau thao tác hơn 30 giây) theo hai hồ sơ low/high, và bản trùng nguyên byte theo tỉ lệ cấu hình. Ground truth (thành thạo ẩn theo thời gian, các episode bế tắc, sự kiện bị chèn muộn/trùng) nằm ở tệp riêng, không đi vào luồng sự kiện. Bế tắc theo định nghĩa của Beck và Gong [3]: từ 10 cơ hội trên một kỹ năng mà chưa từng đúng 3 lần liên tiếp.

Bảng 3.1. Dữ liệu mô phỏng của một run Exp A (1.000 người học, 600 s, hồ sơ trễ low): trung vị (min–max) qua seed. Nguồn: metrics.dataset trong results/raw/runs.jsonl.

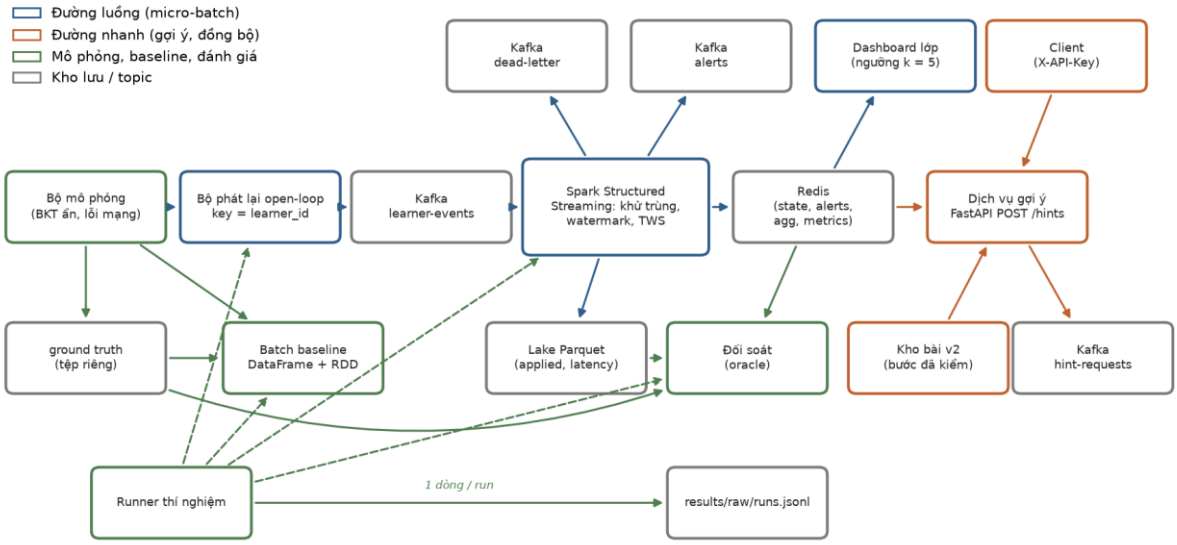
Tốc độ	Sự kiện duy nhất	Tỉ lệ trùng	Tỉ lệ muộn	Episode bế tắc	Người học cellular
100 ev/s	63.020 (62.955–63.234)	1,99% (1,84–2,03)	1,34% (1,30–1,36)	1.549 (1.540–1.616)	407 (406–416)

1.000 ev/s	630.660 (629.542–630.698)	1,99% (1,97–2,02)	1,36% (1,34–1,37)	2.987 (2.913–3.087)	407 (406–416)
10.000 ev/s	6.300.578 (6.297.337–6.300.708)	2,00% (2,00–2,00)	1,35% (1,34–1,37)	2.987 (2.913–3.087)	407 (406–416)

Hạn chế của dữ liệu: hành vi do tham số nhóm tự đặt; sự kiện muộn độc lập từng sự kiện (không mô phỏng mất mạng theo đợt); mỗi người học một loại kết nối cố định. Vì vậy kết quả cho thấy cơ chế và đánh đổi của pipeline, không phải số đo hành vi học thật.

4. Phương pháp và kiến trúc

Hệ thống có hai đường xử lý tách nhau có chủ đích (Hình 4.1). Đường luồng cập nhật trạng thái và phát cảnh báo theo micro-batch; đường nhanh phục vụ yêu cầu gợi ý đồng bộ và chỉ đọc kết quả mà đường luồng đã ghi vào Redis, nên độ trễ của gợi ý không phụ thuộc chu kỳ micro-batch.



Hình 4.1. Kiến trúc: đường luồng (xanh), đường nhanh (cam), mô phỏng, baseline và đánh giá (lục), kho lưu và topic (xám). Mũi tên đứt: runner điều phối từng run.

4.1. Đưa sự kiện vào Kafka

Bộ phát lại gửi mỗi dòng của lịch sự kiện đúng lúc replay_start + offset (open-loop): lịch gửi không phụ thuộc hệ thống nhanh hay chậm, tránh hiện tượng coordinated omission mà bộ sinh tải closed-loop gặp phải [8]. Khóa Kafka là learner_id nên mọi sự kiện của một người học vào cùng partition. Producer bật idempotence nên retry không sinh thêm bản sao; bản trùng duy nhất là bản do bộ mô phỏng chèn có chủ đích. Hết log, bộ phát lại gửi heartbeat vào từng partition để watermark vượt qua cuối log. Tốc

độ gửi thực tế được ghi trong mọi run; run có tốc độ thấp hơn mục tiêu quá 5% bị đánh dấu không hợp lệ.

4.2. Trạng thái theo người học trên Spark

Job Spark kiểm hợp đồng từng bản ghi (bản ghi sai sang topic dead-letter, tách trước watermark để rác không đẩy watermark), đặt watermark W trên event_time, khử trùng theo event_id bằng dropDuplicatesWithinWatermark [13], rồi groupBy(learner_id) với transformWithStateInPandas [12] trên state store RocksDB. Bộ xử lý giữ cho mỗi (người học, kỹ năng) số cơ hội, chuỗi đúng/sai liên tiếp, cờ thành thạo, và phát cảnh báo bế tắc hoặc sai liên tiếp. Ba chế độ xử lý thứ tự:

- **eager:** áp sự kiện ngay khi đến, gắn cờ khi sai thứ tự; độ trễ thấp nhất nhưng trạng thái có thể khác thứ tự event time.
- **ordered:** đệm sự kiện tới khi watermark vượt qua rồi áp đúng thứ tự event time; độ trễ hệ thống không nhỏ hơn W theo thiết kế.
- **none:** không watermark, không timer; khử trùng bằng tập event_id trong state, nên không có cảnh báo không hoạt động; dùng làm mốc cho RQ3.

4.3. Ghi kết quả idempotent và đối soát

State store của Spark là exactly-once, nhưng side effect trong foreachBatch có thể chạy lại khi khôi phục, tức at-least-once. Vì vậy mọi lần ghi đều idempotent: script Lua cập nhật Redis chỉ khi phiên bản (batch_id, state_version) mới hơn, cảnh báo có alert_id tất định và chỉ vào alerts:stream một lần, tệp Parquet của lake ghi đè đúng tệp của batch. Sau mỗi run, bộ đối soát dựng oracle ngoại tuyến từ đúng các sự kiện pipeline đã nhận, áp đúng thứ tự bằng cùng hàm luật, rồi so với Redis; cảnh báo được so với ground truth để tính recall và thời gian phát hiện. Trong thử nghiệm giết job giữa một micro-batch rồi chạy lại từ cùng checkpoint, trạng thái khớp oracle 558/558 (100%), số sự kiện bị áp hai lần là 0 và số cảnh báo trùng là 0.

4.4. Dịch vụ gợi ý có kiểm chứng

Dịch vụ FastAPI không gọi mô hình ngôn ngữ lúc chạy. Khi khởi động, mọi bước giải của kho bài được SymPy kiểm và đánh dấu bước nào lộ đáp án. Mỗi yêu cầu chẩn đoán biểu thức của người học trong tiến trình con có hạn giờ, đọc trạng thái từ Redis, chọn bậc rồi ghép gợi ý: bậc 1 là khái niệm và một bài tương tự của tập dev (tìm bằng chỉ mục đảo); bậc 2 là câu chẩn đoán và dạng của bước tiếp theo với mọi số thay bằng ô □; bậc 3 là bước tiếp theo đã kiểm. Bước lộ đáp án chỉ được đưa sau khi người học sai đáp án cuối hai lần; trước đó bậc 3 chỉ đưa dạng của chính bước đó. Bản nháp được kiểm lần cuối: mọi biểu thức phải đúng nguyên văn một bước đã kiểm hoặc đề của một bài dev. Thiếu trạng thái (Redis lỗi, khóa trống) thì dịch vụ chỉ cho bậc 1 và không bao giờ đưa đáp án (fail-closed). Bộ sinh template (gợi ý mẫu theo kỹ năng, không cá nhân hóa) là nhánh đối chứng.

Để trình diễn, trang Học sinh (Streamlit, chỉ nghe 127.0.0.1) cho một người đóng vai học sinh: chọn bài, nhập từng bước, nhận kết quả kiểm bằng cùng bộ kiểm SymPy, và xin gợi ý qua đúng API trên. Mỗi thao tác là một sự kiện LearnerEvent hợp lệ gửi vào Kafka, nên đi qua cùng job Spark như sự kiện mô phỏng; dịch vụ gợi ý đọc trạng thái của chính người đóng vai từ Redis và dashboard đếm người đó vào lớp. Trang này chỉ dùng cho demo, không tham gia phép đo.

4.5. Batch baseline

Batch chạy theo lịch ngày hoặc giờ trên cùng event log và cùng hàm luật với streaming, chỉ thấy sự kiện đã gửi trước mốc cắt. Có hai bản: DataFrame (applyInPandas) và RDD MapReduce với partition theo learner_id và secondary sort theo (người học, kỹ năng, event_time); hai bản cho kết quả trùng nhau. Một episode bế tắc được coi là phát hiện lúc kết quả của job đầu tiên chứa nó hiển thị, tức mốc cắt cộng thời gian chạy đo được của job.

4.6. Độ phức tạp

- **Bộ nhớ trạng thái:** O(L·K) với L người học và K kỹ năng mỗi người; state khử trùng và hàng đợi của chế độ ordered thêm O(r·W) với r là tốc độ sự kiện và W là watermark.
- **Chi phí mỗi sự kiện:** O(1) cập nhật trạng thái của một khóa (ordered thêm O(log b) để sắp b sự kiện đang đệm); song song hóa theo partition của learner_id.
- **Batch:** mỗi job đọc lại toàn bộ log tới mốc cắt, O(N log N) cho bước sắp xếp, nên độ tươi bị chặn bởi chu kỳ lịch chứ không bởi thời gian tính.
- **Gợi ý:** tra bảng và kiểm bản nháp bằng Python thuần, O(độ dài bài) mỗi yêu cầu; SymPy chỉ chạy lúc khởi động và khi chẩn đoán biểu thức người học (có hạn giờ).

4.7. Tổng hợp theo (người học, bài) bằng MapReduce

Ngoài batch baseline, nhóm hiện thực bản thiết kế MapReduce riêng của nhóm (tổng hợp sự kiện học tập theo người học và bài toán) bằng các phép map, reduceByKey, mapPartitions và reduce của Spark RDD. Đối chiếu với các mục của bản thiết kế:

- **Mục 1, đơn vị dữ liệu và khóa phân vùng:** mỗi dòng đầu vào là một lần giao sự kiện (chuỗi JSON đúng như trên Kafka, gồm cả bản trùng). Khóa trung gian là (learner_id, problem_id); partitioner chỉ theo learner_id (crc32), nên mọi bài của một người học về cùng một reducer. Trường ingest_time của thiết kế được thay bằng sent_time có sẵn trong hợp đồng sự kiện.
- **Mục 2, hợp đồng map:** mỗi bản ghi được kiểm theo hợp đồng LearnerEvent với cùng tên lý do lỗi như dead-letter của streaming, và event_type được chuẩn hóa về mã hành động. Bản ghi lỗi đi sang đầu ra riêng kèm event_id và lý do, không làm mất bản ghi khác. Sự kiện muộn (sent_time − event_time > 30 s) vẫn được cộng vì batch đọc cả log; nó chỉ được đếm. Khử trùng theo event_id dùng một phép gộp

kết hợp và giao hoán (giữ bản gửi sớm nhất), nên combiner và task chạy lại không làm tăng bộ đếm; job không dùng accumulator.

- **Mục 3, gom và hợp nhất:** bộ đếm số sự kiện, lượt thử, lượt sai, lần xin gợi ý (cộng), cờ hoàn thành (OR) và event_time đầu/cuối (min/max) đều gộp được bằng combiner. Top-10 bài bị làm sai nhiều nhất: trước hết gom tổng số sai theo problem_id, sau đó mỗi partition chọn top-10 cục bộ bằng heap và một bước reduce gộp các danh sách; đồng hạng xếp theo problem_id tăng dần. Mỗi run được so chính xác với một bản tham chiếu tuần tự viết lại độc lập (Python thuần, không Spark) và với số đếm của bộ mô phỏng; lệch ở bất kỳ điểm nào thì run bị ghi failed.
- **Mục 4–6, giả thuyết nút thắt, ma trận và minh chứng:** CPU và RSS của driver và từng executor (mẫu 0,5 s), I/O đĩa và byte loopback của cả máy, và từ REST API của Spark: shuffle, spill, task lỗi hoặc chạy lại, run time theo task. Minh chứng nằm dưới results/mapreduce/ (configs, raw_logs, metrics, figures, error_analysis).

Phần không làm theo thiết kế: không có Hadoop, HDFS hay YARN. Job chạy bằng Spark RDD trên một máy, đọc tệp trên ext4 cục bộ bằng textFile (TextInputFormat của Hadoop); điều phối và chạy lại task do scheduler của Spark đảm nhiệm. Hai cấu hình tài nguyên là hai master local-cluster của Spark trên cùng máy, nên shuffle giữa hai executor đi qua loopback chứ không qua mạng vật lý. Nhóm không chủ động giết worker để thử chịu lỗi; chỉ đếm task lỗi và task chạy lại.

5. Thiết lập thí nghiệm

Ma trận cấu hình máy đọc (configs/experiments/plan-v1.0.yaml) gồm 115 run. Cấu hình mặc định: 1.000 người học, 1.000 ev/s, 3 Kafka partition, trigger ASAP, chế độ eager, W = 120 s, tỉ lệ trùng 2%, mô hình người học rule, 600 s mỗi run, loại 60 s khởi động, seed 11, 22, 33. Cấu hình xuất hiện ở nhiều thí nghiệm là một cấu hình, chạy một lần mỗi seed; ablation vì vậy dùng đúng dữ liệu và seed của run gốc.

Bảng 5.1. Ma trận thí nghiệm và tiến độ tại thời điểm dựng báo cáo (nguồn: plan YAML và runs.jsonl).

Thí nghiệm	Trục thay đổi	Cấu hình	Run hợp lệ / kế hoạch
A — tải × partition	tốc độ × partition	9	27/27
C — độ tươi và chi phí (RQ1)	trigger + batch ngày/giờ	6	18/18
D — watermark, sự kiện muộn (RQ3)	chế độ × W × hồ sơ trễ (+2 cấu hình none)	14	42/42
E — API gợi ý (RQ2)	người dùng đồng thời	4	12/12
B — số người học	số người học	3	9/9
Ablation khử trùng	khử trùng	2	6/6
Ablation xử lý muộn	chế độ	2	6/6
Ablation mô hình người học	mô hình người học	2	6/6
Ablation bộ sinh gợi ý	bộ sinh gợi ý	2	6/6

Stress: soak 30 phút	1800 s ở cấu hình mặc định	1	1/1
Stress: trần ingestion	tốc độ × tiến trình gửi	6	6/6

Thí nghiệm mở rộng MapReduce (giao thức §5) có ma trận riêng: 3 quy mô × 2 cấu hình × 3 lần lặp = 18 run, đã đo 18 run. 3 bộ dữ liệu có cùng 0,1 sự kiện/giây mỗi người học trong 1.200 s, hồ sơ muộn low, tỉ lệ gửi trùng 2%, seed 11; thêm 0,1% bản ghi bị làm hỏng có chủ đích và 3 heartbeat để kiểm đường lỗi (100 người học: 12.448 lần giao, 7 MB; 1.000 người học: 125.486 lần giao, 69 MB; 10.000 người học: 1.256.768 lần giao, 692 MB). Config A = 1 worker × 2 lõi × 4.096 MB (local-cluster[1,2,4096]); Config B = 2 worker × 2 lõi × 4.096 MB (local-cluster[2,2,4096]); cả hai dùng 8 partition cho mỗi shuffle, Spark 4.1.3. Mỗi cấu hình dùng một SparkSession cho mọi run của nó; một job khởi động chạy trước và không tính. Một lần lặp là chạy lại job trên cùng dữ liệu, nên khoảng min–max chỉ phản ánh nhiễu của máy.

Mọi run chạy trên một máy: Intel(R) Core(TM) i9-14900K, 24 luồng logic, WSL2 (kernel 6.18.54.1-microsoft-standard-WSL2, giới hạn RAM 25,2 GB); Spark local[*], driver 4g; Python 3.12.3, PySpark 4.1.3, openjdk version "17.0.20.1" 2026-08-18; Kafka 4.3.2 và Redis 8.10.2-alpine trong Docker Compose, image ghim theo digest. Bộ phát lại, Spark, Kafka và Redis cùng chia một máy, nên kết quả đo song song hóa trên một nút, không đo hệ phân tán.

Chỉ số:

- **Độ trễ hệ thống** = lúc trạng thái hiển thị (sau khi mọi lệnh Redis của batch xong) − sent_time; SLO: P95 ≤ 5 s ở 1K ev/s, chế độ eager.
- **Độ cũ trạng thái** = lúc hiển thị − event_time (gồm độ trễ mạng mô phỏng).
- **Thời gian phát hiện bế tắc** = lúc cảnh báo hiện lần đầu − event_time của sự kiện khởi phát episode; batch và streaming dùng cùng mốc.
- **Throughput bền vững:** độ dốc hồi quy của processing lag trong 300 s cuối phần phát lại ≤ 1% tốc độ mục tiêu [8].
- **Độ đúng trạng thái:** tỉ lệ cặp (người học, kỹ năng) mà Redis khớp oracle ở cuối run; oracle áp lại đúng các sự kiện run đã áp, theo thứ tự event time, nên chỉ số này đo lỗi thứ tự, còn sự kiện bị loại đo riêng; tỉ lệ sự kiện bị loại theo loại kết nối là lát cắt công bằng.
- **Gợi ý:** P95 độ trễ API (SLO 300 ms) theo số người dùng đồng thời; hint validity = SymPy đã kiểm ∧ rater chấm đúng ∧ không lộ đáp án.

Phân tích thống kê

Mỗi cấu hình được chạy 3 lần độc lập với seed khác nhau; một lần chạy là đơn vị phân tích. Trong mỗi lần chạy, độ trễ đầu-cuối của từng sự kiện được tóm tắt bằng phân vị 50, 95 và 99 sau khi loại 60 giây khởi động; giữa các lần chạy, nhóm báo cáo trung vị và khoảng min–max của các phân vị này. Vì chỉ có 3 lần chạy mỗi cấu hình, nhóm

không thực hiện kiểm định giả thuyết và chỉ so sánh mô tả. Độ đúng trạng thái được tính so với ground truth của bộ mô phỏng. Gợi ý được đánh giá trên mẫu phân tầng, do hai rater độc lập chấm mù theo rubric; mức đồng thuận được báo bằng Cohen's κ. Phần mềm: Python 3.12.3, PySpark 4.1.3, Kafka 4.3.2.

Audit gợi ý dùng 120 gợi ý phân tầng theo bậc × bộ sinh (cùng số gợi ý mỗi ô) rút từ bộ gợi ý offline của tập test; tỉ lệ báo trên mẫu phân tầng, không có trọng số theo tổng thể. Khoảng tin cậy của κ tính bằng bootstrap phân vị trên gợi ý.

Cách chấm: Hai thành viên nhóm không chấm từ phiếu trống: theo yêu cầu của nhóm, mỗi người nhận phiếu Excel đã điền sẵn nhãn của một lượt chấm thử bằng mô hình (hai lượt độc lập), người chấm 1 nhận lượt 1, người chấm 2 nhận lượt 2; mỗi người đọc từng dòng và sửa nhãn mình không đồng ý, làm độc lập. So với bản điền sẵn, người chấm 1 sửa 0 / 3 / 56 ô nhãn, người chấm 2 sửa 2 / 5 / 25 ô (rà soát kho bài / vòng hiệu chỉnh / audit chính). Kết quả vì vậy là nhãn do người rà và sửa, không phải nhãn chấm từ đầu; κ có thể bị đẩy cao vì cả hai người bắt đầu từ bản nháp (neo theo bản nháp), dù hai bản nháp là hai lượt độc lập. Hai người không thấy nguồn sinh của gợi ý. Sau vòng hiệu chỉnh, hai người siết hai cách đọc nhãn: correct = no khi bài ví dụ hay bài tương tự mà gợi ý bảo người học bắt chước không áp dụng cho bước hiện tại của bài; diagnostic = yes chỉ khi gợi ý gọi đúng loại lỗi người học thực sự mắc (lời khuyên chung như "tính lại cẩn thận" là no). Kết quả vòng hiệu chỉnh 20 gợi ý (không tính vào kết quả): hợp lệ theo cả hai người 12/20 = 60,0% [38,7–78,1%], κ = 1,000.

6. Kết quả

Tại thời điểm dựng báo cáo, ma trận có 115/115 run hợp lệ; ma trận MapReduce có 18/18 lần lặp đo được. Số nào chưa có hiện dấu chỗ trống ghi rõ nguồn; không có số nào được điền tay.

6.1. RQ1 — Độ tươi và chi phí: streaming so với batch

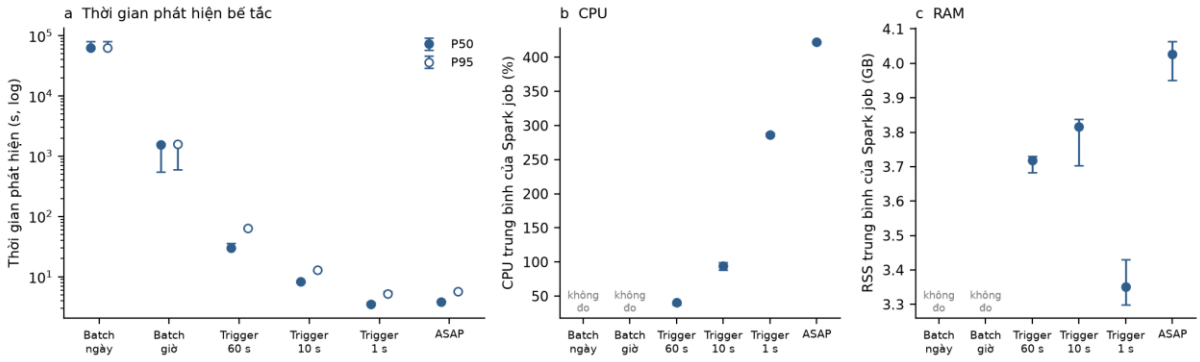
Bảng 6.1. Thời gian phát hiện bế tắc theo độ tươi (Exp C, 1K người học, 1K ev/s). Batch chạy trên đúng cửa sổ log đã phát lại của run streaming cùng seed; batch không ghi CPU nên cột cuối là tổng thời gian chạy các job. Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Cấu hình	Phát hiện P50	Phát hiện P95	Recall bế tắc	CPU Spark (TB)	RAM Spark (TB) / batch
Batch ngày	62.946,7 s (61.267,0–80.976,6)	62.992,5 s (61.313,0–81.023,0)	100,0% (100,0–100,0)	không đo	46,7 s (42,5–47,3) chạy
Batch giờ	1.547,8 s (550,3–1.610,1)	1.594,1 s (596,1–1.656,1)	100,0% (100,0–100,0)	không đo	44,0 s (43,0–68,4) chạy
Trigger 60 s	30,2 s (30,2–36,2)	64,1 s (63,7–64,2)	97,8% (97,7–98,6)	40% (37–41)	3,72 GB (3,68–3,73)
Trigger 10 s	8,3 s (7,9–8,5)	12,8 s (12,8–12,8)	97,6% (97,0–98,1)	95% (88–100)	3,82 GB (3,70–3,84)

Trigger 1 s	3,5 s (3,4–3,7)	5,3 s (5,2–5,7)	96,8% (96,6–97,4)	286% (285–289)	3,35 GB (3,30–3,43)
ASAP	3,8 s (3,8–4,0)	5,7 s (5,5–5,9)	97,2% (96,7–97,5)	422% (420–424)	4,03 GB (3,95–4,06)

Trung vị thời gian phát hiện của batch theo ngày là 62.947 s (61.267–80.977), của streaming trigger ASAP là 3,84 s (3,81–4,03); tỉ số trung vị là 16.388× (15.195–21.237). Batch theo giờ là 1.548 s (550–1.610). Với batch, thời gian phát hiện bị chặn dưới bởi khoảng cách từ lúc khởi phát tới mốc cắt kế tiếp, không bởi thời gian tính; với streaming, nó bị chặn bởi chu kỳ trigger cộng thời gian một micro-batch. Cái giá của streaming là một tiến trình Spark chạy liên tục (cột CPU và RAM), trong khi batch chỉ chiếm máy trong thời gian chạy job.

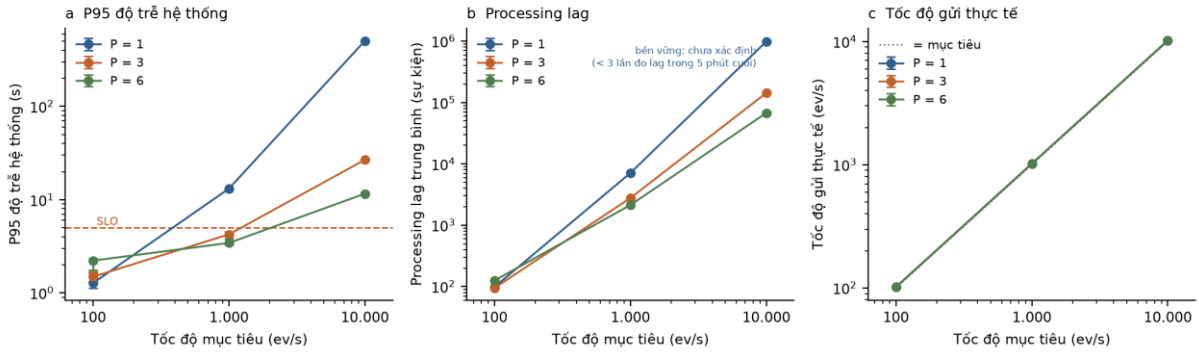
Đối chiếu (đo trong giai đoạn phát triển, ngoài ma trận, log mô phỏng 1K người học × 1 ngày): batch theo ngày phát hiện đủ episode (recall 100,0%) với trung vị 8,1 giờ sau khởi phát, theo giờ 30,3 phút (run batch-daily-20261027T113203Z-b48aa8, batch-hourly-20261027T113236Z-e404ea). Số này không dùng cho kết luận RQ1 vì khác cửa sổ log.



Hình 6.1 (H2). a: thời gian phát hiện bế tắc (thang log) từ batch theo ngày tới streaming ASAP; b, c: CPU và RSS trung bình của tiến trình Spark. Điểm = trung vị qua seed, thanh = min–max; loại 60 s khởi động.

6.2. Mở rộng theo tải, partition và số người học

Ở tải mặc định 1K ev/s, P95 độ trễ hệ thống là 4,25 s (4,07–4,54) so với SLO 5 s. Tốc độ bền vững cao nhất quan sát được là 10.000 ev/s (lag không tăng nhưng vượt SLO; cao nhất trong SLO: 1.000 ev/s). Nhận định 'đạt SLO 5 s ở 1.000 ev/s với P = 3 hoặc 6' chỉ đúng cho các run 10 phút: chạy 30 phút (soak) thì cấu hình mặc định không giữ được SLO. Một máy nghĩa là tăng partition chỉ đo được song song hóa trên cùng CPU, không đo được mở rộng ngang. Số liệu đầy đủ theo tốc độ × partition và theo số người học: Bảng E.1, E.2 (phụ lục).

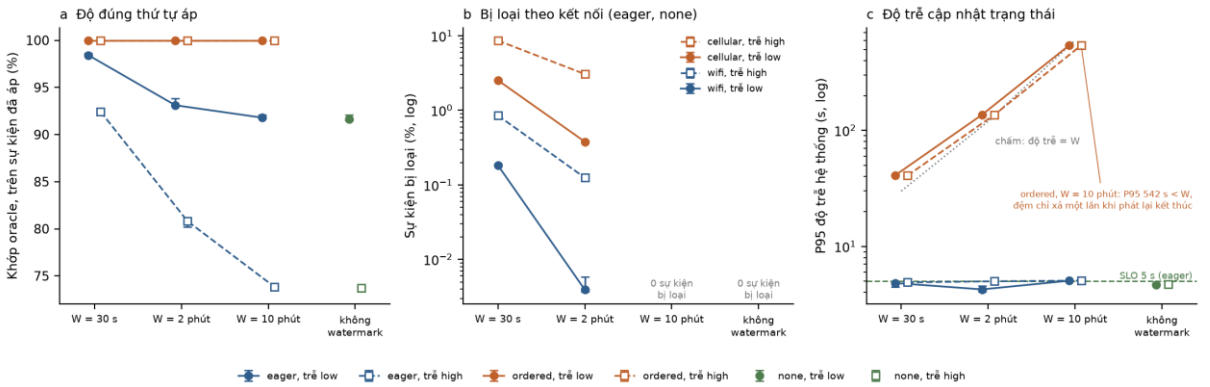


Hình 6.2 (H3). a: P95 độ trễ hệ thống theo tốc độ gửi và số partition (đường đứt: SLO); b: processing lag trung bình; c: tốc độ gửi thực tế so với mục tiêu (đường chấm).

Exp B giữ 1.000 ev/s, P = 3 và tăng số người học: P95 độ trễ hệ thống 1,78 s (1,76–1,80) ở 100, 4,25 s (4,07–4,54) ở 1.000, 38,08 s (37,92–38,40) ở 10.000 người học; ở 10.000 người học 0/3 run đạt SLO 5 s. Ở 10.000 người học, 3/3 run có lag không tăng nhưng lag trung bình đứng ở 20.681 (20.570–20.867) sự kiện, nên P95 vượt SLO dù tốc độ vẫn theo kịp: hàng đợi đứng yên ở mức cao, nhiều khả năng vì mỗi micro-batch phải chạm nhiều khóa trạng thái hơn. Khớp oracle giảm khi ít người học hơn (100 người học: 79,7% (78,5–82,4); 1.000 người học: 93,1% (93,1–93,9); 10.000 người học: 97,2% (97,2–97,2)). Cùng tốc độ gửi, ít người học nghĩa là nhiều sự kiện hơn trên mỗi cặp (người học, kỹ năng), nên một sự kiện muộn dễ bị sự kiện sau vượt hơn trong chế độ eager; đây là giải thích phù hợp với cơ chế, nhóm chưa kiểm riêng. CPU trung bình của Spark: 100 người học: 354% (353–354); 1.000 người học: 422% (420–424); 10.000 người học: 450% (450–452); RSS trung bình 3,86 GB (3,73–3,99) ở 100 và 4,67 GB (4,44–4,95) ở 10.000 người học (Bảng E.2).

6.3. RQ3 — Watermark, sự kiện muộn và công bằng

Với hồ sơ trễ high và W = 2 phút, trạng thái khớp oracle 80,8% (80,2–80,8) ở chế độ eager và 100,0% (100,0–100,0) ở chế độ ordered. Oracle được tính lại trên chính các sự kiện mỗi run đã áp, theo thứ tự event time, nên chỉ số này đo lỗi do áp sai thứ tự; sự kiện bị loại được đo riêng bằng tỉ lệ bị loại. Theo thiết kế, ordered đệm sự kiện tới khi watermark vượt qua nên áp đúng thứ tự event time, đổi lại độ trễ hệ thống xấp xỉ W cộng thời gian vài micro-batch; khi W dài hơn phần phát lại, đệm chỉ được xả lúc phát lại kết thúc (Bảng E.3). Sự kiện đến sau watermark bị loại trước bộ xử lý; ở W = 30 giây (trễ high, eager) tỉ lệ bị loại là 8,64% (8,58–8,70) với người học dùng cellular và 0,85% (0,82–0,86) với wifi (cellular cao hơn wifi). Chênh lệch này, nếu có, đến từ cách xử lý sự kiện muộn chứ không từ người học, nên lát cắt theo loại kết nối được báo cùng chỉ số độ đúng.



Hình 6.3 (H4). a: tỉ lệ trạng thái khớp oracle (tính trên sự kiện đã áp) theo watermark, chế độ và hồ sơ trễ; b: tỉ lệ sự kiện bị loại theo loại kết nối (chế độ eager và none, trục log; ô không loại sự kiện nào ghi chú tại chỗ); c: P95 độ trễ hệ thống (trục log; đường đứt: SLO 5 s của chế độ eager; đường chấm: độ trễ = W).

6.4. RQ2 — Gợi ý đúng, có ích và đủ nhanh

Bảng 6.2. Audit 120 gợi ý tập test, hai thành viên nhóm rà và sửa phiếu điền sẵn, mù nguồn sinh (cách chấm: mục 5; nguồn: docs/hint-audit/summary.md).

Nhãn	Cả hai rater 'yes' [Wilson 95%]	Đồng thuận thô	Cohen's κ
Correct	79/120 = 65,8% [57,0–73,7%]	90,8%	0,781
NotRevealing	120/120 = 100,0% [96,9–100,0%]	100,0%	không xác định
Suggestion	56/120 = 46,7% [38,0–55,6%]	90,0%	0,802
Diagnostic	51/120 = 42,5% [34,0–51,4%]	75,8%	0,518
Positive	10/120 = 8,3% [4,6–14,7%]	100,0%	1,000
Hint validity	79/120 = 65,8% [57,0–73,7%]	90,8%	0,781

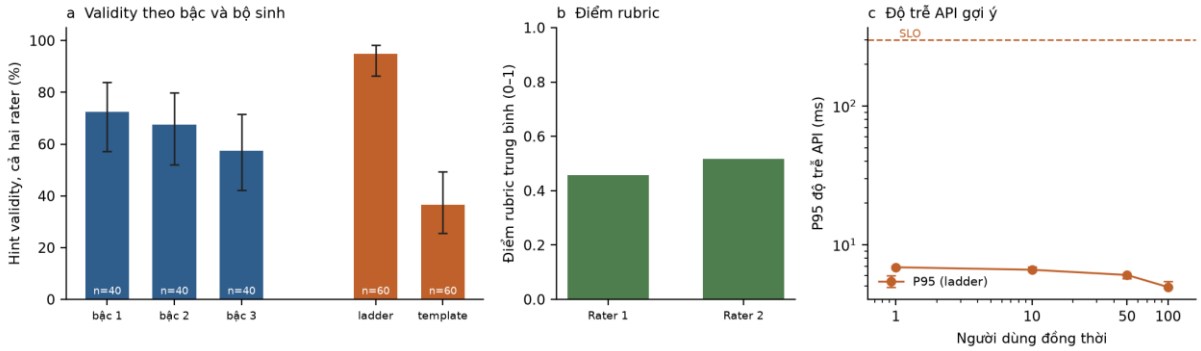
Kiểm máy SymPy đạt 120/120 = 100,0% [96,9–100,0%]. Hint validity theo cả hai rater là 79/120 = 65,8% [57,0–73,7%], κ = 0,781 (bootstrap 95%: 0,643–0,898). Bộ sinh theo bài (ladder) đạt 57/60 = 95,0% [86,3–98,3%], bộ mẫu chung (template) 22/60 = 36,7% [25,6–49,3%]. Cả hai rater đều chấm mọi gợi ý là không lộ đáp án, nên hint validity trùng với nhãn Correct và khác biệt giữa hai bộ sinh nằm ở nhãn này (đúng toán và đúng ngữ cảnh của bài). Nhãn Diagnostic trên các gợi ý có biểu thức sai đạt 42/90 = 46,7% [36,7–56,9%]; nhãn Positive đạt 10/120 = 8,3% [4,6–14,7%]. Điểm rubric: Rater 1: trung bình 0,457, trung vị 0,60; Rater 2: trung bình 0,518, trung vị 0,60. Validity theo từng tầng: Bảng E.4. Các nhãn là nhãn do người rà và sửa từ phiếu điền sẵn (mục 5), nên κ cần đọc cùng giới hạn neo theo bản nháp ở mục 7.3.

Bảng 6.3. Exp E: độ trễ API gợi ý (bộ sinh ladder, mỗi người dùng 1 yêu cầu/giây, SLO P95 ≤ 300 ms). Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Người dùng	P50	P95	P99	Yêu cầu/s	Đạt SLO
1	4,23 ms (2,95–4,33)	6,86 ms (6,70–7,00)	11,04 ms (7,41–19,48)	1,0 (1,0–1,0)	3/3
10	2,99 ms (2,21–3,47)	6,58 ms (6,48–6,89)	7,41 ms (7,33–8,43)	10,0 (10,0–10,0)	3/3

50	2,08 ms (2,03–2,22)	6,03 ms (5,67–6,04)	7,94 ms (7,28–8,01)	50,0 (50,0–50,0)	3/3
100	1,72 ms (1,60–1,72)	4,93 ms (4,93–5,42)	7,25 ms (6,89–8,46)	100,0 (100,0–100,0)	3/3

Đối chiếu (đo trong giai đoạn phát triển, ngoài ma trận, 60 s mỗi mức): P95 = 2,54 ms ở 100 người dùng, 0 phản hồi lỗi (results/raw/hint-load-20261027T132710Z.json).



Hình 6.4 (H5). a: hint validity (cả hai rater, thanh = Wilson 95%) theo bậc và bộ sinh; b: điểm rubric trung bình theo rater; c: P95 độ trễ API theo số người dùng đồng thời (đường đứt: SLO).

6.5. Chạy dài và trần ingestion

Soak chạy cấu hình mặc định trong 30 phút (một seed): P95 độ trễ hệ thống 5,04 s (n = 1) (đạt SLO cả run: 0/1 run), 54,7% (n = 1) số cửa sổ một phút có P95 vượt 5 s, RSS driver tăng 108 MB/phút (n = 1) (5 phút cuối / 5 phút đầu: 1,73× (n = 1)). Độ dốc lag 0,20 (n = 1) ev/s, nên tốc độ vẫn theo kịp (bền vững: 1/1 run). So với các run 10 phút cùng cấu hình (P95 4,25 s (4,07–4,54), 1,9% (1,4–2,9) cửa sổ một phút vượt SLO), độ trễ đuôi cao hơn khi chạy lâu. Nhận định 'đạt SLO 5 s ở 1.000 ev/s với P = 3 hoặc 6' chỉ đúng cho các run 10 phút: chạy 30 phút (soak) thì cấu hình mặc định không giữ được SLO. RSS của driver tăng từ 3,74 GB (n = 1) (trung vị 5 phút đầu) lên 6,46 GB (n = 1) (5 phút cuối), đỉnh 6,48 GB (n = 1). Nhóm ghi đây là rủi ro quan sát được khi chạy dài; một run 30 phút không đủ để nói nguyên nhân (rò bộ nhớ, state store lớn dần hay heap JVM giãn tới giới hạn). Khớp oracle của soak là 93,1% (n = 1), so với 93,1% (93,1–93,9) ở run 10 phút.

Trần ingestion đo riêng bộ phát lại + Kafka một nút (không qua Spark), mỗi ô một run 60 giây với 10.000 người học: 6/6 ô giữ được tốc độ mục tiêu; cao nhất đạt 60.255 ev/s (mục tiêu 60.000 ev/s, 4 tiến trình gửi), nên trần nằm trên mức cao nhất đã thử (60.000 ev/s). Độ trễ lịch gửi P99 (slip) của bộ phát lại nằm trong khoảng 8,2–12,4 ms và CPU trung bình của Kafka 18–48% qua các ô (Bảng E.5). Như vậy ở dải đã thử, giới hạn tốc độ của pipeline nằm ở Spark (mục 6.2) chứ không ở khâu đưa sự kiện vào Kafka.

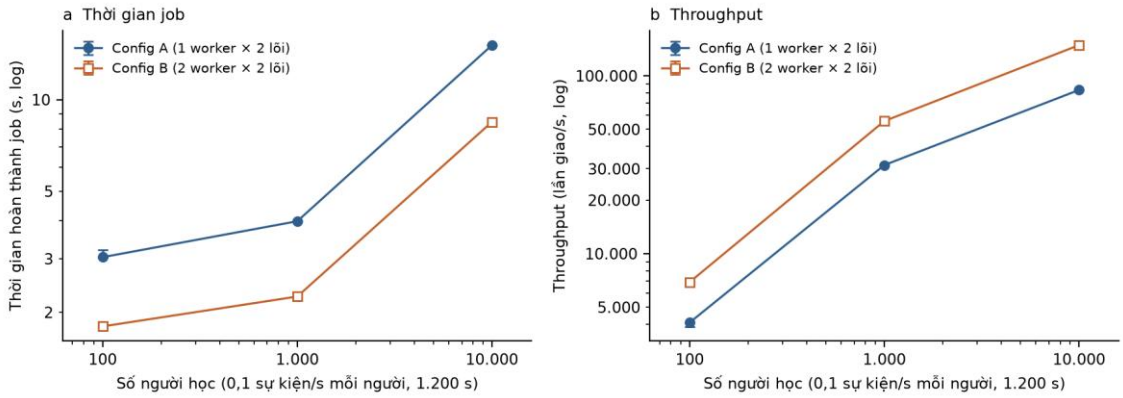
6.6. Thí nghiệm mở rộng MapReduce

Bảng 6.4. Thời gian hoàn thành job và throughput theo quy mô × cấu hình. A/B = tỉ số trung vị thời gian job của Config A trên Config B ở cùng quy mô (min–max = tỉ số các cực trị). 'Khớp tham chiếu' = số lần lặp khớp chính xác bản tham chiếu tuần tự và số đếm của bộ mô phỏng. Ô ghi trung vị (min–max) qua các lần lặp trên cùng dữ liệu. Nguồn:

results/mapreduce/metrics/<run_key>.json; chỉ tính lần lặp có run record batch-mapreduce success trong results/raw/runs.jsonl.

Kịch bản · người học · cấu hình	Lần giao	Thời gian job	Trung bình (s)	Throughput (lần giao/s)	A/B	Khớp tham chiếu
s1 · 100 · A	12.448	3,04 s (3,02–3,21)	3,09	4.100 (3.882–4.118)	—	3/3
s2 · 100 · B	12.448	1,80 s (1,79–1,83)	1,81	6.923 (6.806–6.943)	1,69× (1,65–1,79)	3/3
s3 · 1.000 · A	125.486	3,98 s (3,97–4,01)	3,99	31.490 (31.286–31.632)	—	3/3
s4 · 1.000 · B	125.486	2,25 s (2,23–2,31)	2,27	55.673 (54.370–56.146)	1,77× (1,72–1,79)	3/3
s5 · 10.000 · A	1.256.768	15,10 s (15,04–15,16)	15,10	83.241 (82.900–83.567)	—	3/3
s6 · 10.000 · B	1.256.768	8,45 s (8,40–8,58)	8,48	148.730 (146.476–149.633)	1,79× (1,75–1,80)	3/3

Từ 100 lên 10.000 người học, số lần giao tăng 101 lần, thời gian job tăng 5,0× (4,7–5,0) ở Config A và 4,7× (4,6–4,8) ở Config B, nên throughput tăng theo quy mô: ở quy mô nhỏ, chi phí cố định của mỗi lần chạy (5 job Spark nối tiếp, mỗi job vài stage, cộng khởi động tác vụ Python) chiếm phần lớn thời gian. Config B nhanh hơn A theo tỉ số trung vị thời gian job: 100 người học: 1,69× (1,65–1,79); 1.000 người học: 1,77× (1,72–1,79); 10.000 người học: 1,79× (1,75–1,80), thấp hơn tỉ lệ số lõi (2 lần).



Hình 6.5 (H6). Ma trận MapReduce: a: thời gian hoàn thành job, b: throughput theo số người học, Config A so với B (hai trục log). Điểm = trung vị qua 3 lần lặp, thanh = min–max.

Bảng 6.5. Tài nguyên theo giả thuyết H1–H4 của thiết kế. CPU tính theo % một lõi, cộng qua các executor; RSS executor là tổng đỉnh RSS của các executor (JVM và tiến trình Python con); I/O đĩa đo trên cả máy trong lúc job chạy; shuffle và spill lấy từ REST API của Spark. Ô ghi trung vị (min–max) qua các lần lặp trên cùng dữ liệu. Nguồn: results/mapreduce/metrics/<run_key>.json; chỉ tính lần lặp có run record batch-mapreduce success trong results/raw/runs.jsonl.

Kịch bản	CPU executor (TB)	CPU driver (TB)	RSS đỉnh executor	RSS đỉnh driver	Ghi đĩa (máy)	Shuffle ghi	Shuffle đọc từ executor khác	Spill bộ nhớ / đĩa
s1 · 100 · A	201% (195–201)	27% (24–32)	0,69 GB (0,68–0,69)	0,79 GB (0,71–0,81)	2,1 MB (1,4–693,9)	1,2 MB (1,2–1,2)	0,0 MB (0,0–0,0)	0 MB (0–0) / 0 MB (0–0)
s2 · 100 · B	368% (354–413)	39% (32–53)	1,45 GB (1,44–1,48)	0,76 GB (0,76–0,77)	1,8 MB (1,7–2,3)	1,2 MB (1,2–1,2)	1,2 MB (1,2–1,2)	0 MB (0–0) / 0 MB (0–0)

s3 · 1.000 · A	193% (186–194)	11% (10–15)	1,04 GB (0,73–1,08)	0,85 GB (0,83–0,85)	2,2 MB (2,1–4,6)	12,8 MB (12,8–12,8)	0,0 MB (0,0–0,0)	0 MB (0–0) / 0 MB (0–0)
s4 · 1.000 · B	356% (353–369)	31% (22–31)	1,55 GB (1,54–1,55)	0,79 GB (0,77–0,80)	2,6 MB (2,4–3,1)	12,8 MB (12,8–12,8)	12,7 MB (12,7–12,8)	0 MB (0–0) / 0 MB (0–0)
s5 · 10.000 · A	208% (207–208)	5% (3–5)	1,46 GB (1,43–1,66)	0,87 GB (0,86–0,88)	9,9 MB (5,5–306,7)	132,8 MB (132,8–132,8)	0,0 MB (0,0–0,0)	0 MB (0–0) / 0 MB (0–0)
s6 · 10.000 · B	389% (385–410)	9% (9–10)	3,27 GB (2,81–3,45)	0,81 GB (0,80–0,84)	7,4 MB (7,1–19,5)	132,7 MB (132,7–132,7)	132,7 MB (132,6–132,8)	0 MB (0–0) / 0 MB (0–0)

- **H1, CPU:** ở 10.000 người học, CPU trung bình cộng qua các executor là 208% (207–208) với Config A (1 × 2 lõi) và 389% (385–410) với Config B (2 × 2 lõi), tức 104% và 97% số lõi được cấp; driver dùng 5% (3–5) (A) và 9% (9–10) (B). Job bị giới hạn bởi CPU của executor, phù hợp với giả thuyết H1.
- **H2, RAM:** tổng đỉnh RSS của executor là 1,46 GB (1,43–1,66) (A) và 3,27 GB (2,81–3,45) (B), của driver 0,87 GB (0,86–0,88) (A) ở 10.000 người học; spill bộ nhớ 0 MB (0–0), spill đĩa 0 MB (0–0) (A). Không lần lặp nào spill hay có task lỗi, nên RAM không phải nút thắt ở các quy mô này.
- **H3, đĩa:** đọc đĩa của cả máy bằng 0 byte trong mọi lần lặp, nhiều khả năng vì đầu vào đã nằm trong page cache của hệ điều hành; ghi đĩa 1,8–9,9 MB (trung vị theo kịch bản). Trên máy này I/O đĩa không phải nút thắt, nhưng phép đo không có trường hợp đọc nguội từ đĩa.
- **H4, shuffle:** shuffle ghi 132,8 MB (132,8–132,8) ở 10.000 người học (A). Ở Config B, 132,7 MB (132,6–132,8) trong 265,4 MB (265,3–265,4) shuffle đọc đến từ executor khác (tỉ số 0,50 (0,50–0,50)), tổng thời gian chờ fetch 0 ms (0–1). Byte loopback của cả máy là 1.881 MB (1.881–1.881) ở A, lớn hơn nhiều lượng shuffle vì gồm cả dữ liệu giữa JVM executor và tiến trình Python worker (PySpark trao đổi qua socket cục bộ); vì vậy H4 được đọc bằng số đo shuffle của Spark chứ không bằng loopback, và mạng vật lý không được đo.

H5, lệch tải: ở stage reduce theo learner_id (8 task), tỉ số max/trung vị của run time theo task là 1,07–1,58× (trung vị theo kịch bản); task chậm nhất 70–241 ms so với trung vị 66–201 ms; tỉ số lớn nhất ở các stage đọc shuffle là 1,19–1,79×. Chênh lệch tuyệt đối nhỏ nên khó tách khỏi nhiễu lập lịch. Số bản ghi theo task không đo được lệch khóa: với RDD của PySpark, Spark đếm các lô đã tuần tự hóa chứ không đếm từng khóa (tổng 7.539 bản ghi shuffle đọc so với 156.466 khóa đầu ra ở Config A, 10.000 người học), nên tỉ số max/trung vị của số bản ghi (1,00–1,00×) không nói gì về số khóa; số khóa theo partition không được job ghi lại. Lệch ở khóa problem_id thì có: ở 10.000 người học, bài đứng đầu top-K (P-NTE-058) có 40.151 lượt sai, gấp 8,0 lần bài thứ hai (5.046 lượt). Khóa này không làm lệch reducer vì bước gom theo problem_id có combiner, mỗi partition chỉ gửi đi một tổng cục bộ.

6.6.1. Phân tích lỗi

Bảng 6.6. Chất lượng dữ liệu: giá trị job đếm được / giá trị kỳ vọng (manifest của bộ mô phỏng; bản ghi hỏng và heartbeat là số đã chèn). Mọi lần lặp của cùng quy mô cho cùng số đếm. Nguồn: metrics/<run_key>.json (counts, data_checks), tổng hợp ở results/mapreduce/error_analysis/.

Người học	Lần giao	Sự kiện duy nhất	Bản trùng	Sự kiện muộn	Bản ghi hỏng	Heartbeat	event_id xung đột; khóa đầu ra
100	12.448	12.198 / 12.198	239 / 239	132 / 132	8 / 8	3 / 3	0; 1.561
1.000	125.486	122.887 / 122.887	2.471 / 2.471	1.641 / 1.641	125 / 125	3 / 3	0; 15.665
10.000	1.256.768	1.230.985 / 1.230.985	24.519 / 24.519	16.085 / 16.085	1.261 / 1.261	3 / 3	0; 156.466

18/18 lần lặp khớp chính xác bản tham chiếu (số đếm theo loại, lý do lỗi, từng khóa bộ đếm, tổng bộ đếm, top-K) và khớp số đếm của bộ mô phỏng; các lần lặp của cùng quy mô cho cùng số đếm. Task lỗi: 0, task chạy lại: 0, stage chạy lại: 0, job lỗi: 0; vì không worker nào bị giết, đường chạy lại task không được thử trong ma trận này (phép gộp khử trùng kết hợp và giao hoán được kiểm bằng test đơn vị). Lần lặp bị loại khỏi bảng: không có.

Ở 10.000 người học, job phát hiện 1.261 bản ghi hỏng (đã chèn 1.261), chia theo lý do: event_time 150, event_type 148, learner_id 146, learning_fields 138, malformed_json 131, missing_field 130, payload 146, sent_before_event 149, unknown_field 123; 1.261 bản ghi trong số đó vẫn giữ được event_id (kể cả JSON bị cắt, nhờ tìm event_id trên chuỗi gốc). Ở mọi quy mô, số bản ghi hỏng phát hiện được đúng bằng số đã chèn. Số sự kiện duy nhất và số bản trùng khớp manifest của bộ mô phỏng ở mọi quy mô; số event_id trùng mà khác nội dung (tổng mọi lần lặp): 0. Sự kiện muộn được đếm khớp manifest và vẫn được cộng vào bộ đếm. Mẫu bản ghi lỗi kèm lý do nằm trong results/mapreduce/error_analysis/invalid_samples.jsonl.

7. Ablation, phân tích lỗi và các mối đe dọa tính hợp lệ

7.1. Ablation

Bảng 7.1. Mỗi ablation đổi đúng một yếu tố quanh cấu hình mặc định, cùng dữ liệu và seed. Hàng bộ sinh gợi ý: cột ba là hint validity của audit (cả hai rater), cột cuối là P95 API. Trung vị (min–max) qua seed.

Yếu tố	Biến thể	Khớp oracle / validity	Recall bế tắc	P95 độ trễ
Mặc định	eager, khử trùng, luật	93,1% (93,1–93,9)	97,2% (96,7–97,5)	4,25 s (4,07–4,54)
Khử trùng	tắt khử trùng	21,1% (20,9–21,2)	81,6% (80,9–83,7)	4,63 s (4,52–4,65)
Xử lý muộn	ordered (đệm tới W)	100,0% (100,0–100,0)	99,6% (99,3–99,6)	136,85 s (136,53–137,10)
Mô hình người học	BKT	93,2% (93,1–93,8)	96,7% (96,7–97,5)	5,02 s (4,93–5,08)
Bộ sinh gợi ý	ladder (10 người dùng)	57/60 = 95,0% [86,3–	—	6,58 ms (6,48–6,89)

		98,3%]		
Bộ sinh gợi ý	template (10 người dùng)	22/60 = 36,7% [25,6–49,3%]	—	6,53 ms (6,33–6,61)

Bảng chỉ so sánh mô tả quanh cấu hình mặc định (khớp oracle 93,1% (93,1–93,9), recall bế tắc 97,2% (96,7–97,5), P95 4,25 s (4,07–4,54)). Tắt khử trùng làm khớp oracle còn 21,1% (20,9–21,2) và recall bế tắc còn 81,6% (80,9–83,7): bản gửi trùng được áp hai lần nên bộ đếm cơ hội và chuỗi đúng/sai lệch khỏi oracle (oracle áp mỗi event_id một lần); đồng thời không sự kiện nào bị loại, vì trong job này bước loại sự kiện sau watermark nằm ở toán tử khử trùng. Ordered áp đúng thứ tự (khớp oracle 100,0% (100,0–100,0)) nhưng P95 lên 136,85 s (136,53–137,10) (0/3 run đạt SLO) vì sự kiện phải chờ watermark. BKT thay luật đếm cho khớp oracle 93,2% (93,1–93,8) (oracle dùng cùng mô hình), P95 5,02 s (4,93–5,08) và 1/3 run đạt SLO, so với 3/3 ở cấu hình mặc định. Ở bộ sinh gợi ý, P95 API của template (6,53 ms (6,33–6,61)) và ladder (6,58 ms (6,48–6,89)) ở 10 người dùng (SLO 300 ms; đạt SLO: ladder 3/3, template 3/3 run), nên khác biệt cần xét giữa hai bộ sinh nằm ở chất lượng gợi ý; hint validity theo bộ sinh lấy từ audit của hai thành viên nhóm (57/60 = 95,0% [86,3–98,3%] so với 22/60 = 36,7% [25,6–49,3%]).

7.2. Phân tích lỗi

- **Cảnh báo bế tắc so với oracle:** ở cấu hình mặc định, mỗi run có 2.976 (2.909–3.084) khởi phát theo oracle, thiếu 291 (271–314) và thừa 268 (250–302) cảnh báo. Thành thạo theo luật khớp thành thạo ẩn của bộ mô phỏng 100,0% (100,0–100,0): luật đếm là xấp xỉ của trạng thái ẩn, không phải nó.
- **Kho bài:** 4 trên 100 bài rà soát bị ít nhất một trong hai thành viên chấm không đạt; lý do từng bài nằm trong docs/manual-review/round-2/summary.md. Bộ kiểm tự động không bắt được loại lỗi này.
- **Gợi ý:** tầng có hint validity thấp nhất là bậc 3 (23/40 = 57,5% [42,2–71,5%]); chi tiết từng gợi ý bị chấm 'no' kèm ghi chú của rater nằm trong docs/hint-audit/summary.md.
- **Job MapReduce:** 18/18 lần lặp khớp bản tham chiếu và số đếm của bộ mô phỏng, 0 task lỗi; chất lượng dữ liệu, lý do của bản ghi hỏng và độ lệch theo task: mục 6.6.1 và results/mapreduce/error_analysis/.

7.3. Các mối đe dọa tính hợp lệ

- **Dữ liệu mô phỏng.** Hành vi người học, độ trễ mạng và tỉ lệ muộn/trùng do nhóm tự đặt; ground truth chính xác theo định nghĩa của bộ mô phỏng. Kết quả cho thấy cơ chế và đánh đổi của pipeline, không đo hiệu quả học tập hay thiên lệch ngoài thực tế.

- **Một máy.** Bộ phát lại, Spark, Kafka và Redis chia chung CPU và bộ nhớ; tăng partition đo song song hóa trên một nút. Mỗi cấu hình chỉ có 3 seed nên nhóm không dùng kiểm định giả thuyết.
- **Giới hạn của chấm mù trong audit gợi ý.** Rater không thấy nguồn sinh, nhưng văn phong để lộ một phần: gợi ý ladder bậc 1 nêu mã một bài tương tự, bậc 2 và bậc 3 bị khóa đưa dạng bước có ô □. Rater có thể đoán bộ sinh từ các dấu hiệu này.
- **Thay đổi bộ sinh sau vòng chấm thử.** Sau một vòng chấm thử bằng mô hình trên phiếu hiệu chỉnh (20 gợi ý) và trước khi rút phiếu vòng chính, bậc 3 của ladder khi bước tiếp theo lộ đáp án được đổi từ câu mẫu chung sang dạng bước của chính bài (commit 884280f). Rubric, định nghĩa validity, cỡ mẫu và thiết kế phân bổ không đổi; vòng hiệu chỉnh không gộp vào vòng chính.
- **Phiếu chấm điền sẵn bản nháp.** Hai thành viên nhóm không chấm từ phiếu trống: theo yêu cầu của nhóm, mỗi người nhận phiếu Excel đã điền sẵn nhãn của một lượt chấm thử bằng mô hình (hai lượt độc lập), người chấm 1 nhận lượt 1, người chấm 2 nhận lượt 2; mỗi người đọc từng dòng và sửa nhãn mình không đồng ý, làm độc lập. So với bản điền sẵn, người chấm 1 sửa 0 / 3 / 56 ô nhãn, người chấm 2 sửa 2 / 5 / 25 ô (rà soát kho bài / vòng hiệu chỉnh / audit chính). Kết quả vì vậy là nhãn do người rà và sửa, không phải nhãn chấm từ đầu; κ có thể bị đẩy cao vì cả hai người bắt đầu từ bản nháp (neo theo bản nháp), dù hai bản nháp là hai lượt độc lập. Bản chấm thử bằng mô hình vẫn lưu riêng trong các thư mục model-prerating/, không phải kết quả chấm của người và không gộp vào kết quả; kế hoạch ban đầu là hai người chấm từ phiếu trống và không mở bản này trước khi nộp.
- **Ý nghĩa cờ dirty.** Trong run record, dirty chỉ xét các đường dẫn mã (src, scripts, configs, contracts, tệp build); các bản ghi phát triển cũ hơn tính dirty trên cả repo nên luôn là true vì runner tự ghi results/raw. Commit của các run thuộc ma trận: 1c35b23: 14 run, dirty = false; 1c35b23: 1 run, dirty = true; 1cc939c: 16 run, dirty = false; 3d4d04c: 1 run, dirty = false; 5b38433: 1 run, dirty = true; 640a4e5: 2 run, dirty = false; 736e0c7: 2 run, dirty = false; 9579ae9: 3 run, dirty = false; bf1bd25: 2 run, dirty = false; c9c0bfc: 9 run, dirty = false; d2d4855: 1 run, dirty = false; d2d4855: 1 run, dirty = true; dea3234: 60 run, dirty = false; e5fb407: 3 run, dirty = false; e5fb407: 1 run, dirty = true; f7dc1ac: 1 run, dirty = false. Ma trận có thể chạy qua nhiều commit; dirty = false chỉ nói mã khớp commit ghi trong bản ghi, nên khác biệt giữa các run được đối chiếu bằng lịch sử commit chứ không bằng cờ này.
- **Run bị thay thế.** Hai run Exp A đầu tiên chạy nhầm kho bài mini của bộ mô phỏng; ma trận được dừng, sửa và chạy lại từ đầu. Các bản ghi đó giữ trong runs.jsonl (chỉ ghi nối) nhưng không khớp tiền tố cấu hình nào nên bị loại: st-l1000-r100-p1-rp2-eager-w120-low-dup2-rule-tasap-600s-s11-20261027T141218Z, st-l1000-r100-p1-rp2-eager-w120-low-dup2-rule-tasap-600s-s22-20261027T142326Z.

- **Lần thử không thành công.** Theo quy tắc resume, mỗi seed lấy run success cuối cùng; các lần thử sau đây giữ trong runs.jsonl nhưng không vào kết quả: st-l1000-r1000-p1-rp2-eager-w120-low-dup2-rule-tasap-600s-pbv2test-s11-20261027T162620Z (aborted); st-l1000-r1000-p3-rp2-eager-w120-low-dup2-nodedup-rule-tasap-600s-pbv2test-s11-20261028T081224Z (timeout); st-l1000-r1000-p3-rp2-eager-w120-low-dup2-nodedup-rule-tasap-600s-pbv2test-s11-20261028T083245Z (aborted). Lần hết giờ của ablation tắt khử trùng do bộ lọc heartbeat bị optimizer đẩy xuống dưới watermark khi không còn toán tử khử trùng, nên watermark không vượt cuối log; các run được dùng chạy ở commit 1cc939c, d2d4855, dea3234, sau bản sửa.
- **Thời lượng run.** Các cấu hình của ma trận chạy 600 s; soak và trần ingestion chỉ có một run mỗi ô nên không có khoảng min–max. Nhận định 'đạt SLO 5 s ở 1.000 ev/s với P = 3 hoặc 6' chỉ đúng cho các run 10 phút: chạy 30 phút (soak) thì cấu hình mặc định không giữ được SLO.
- **Thí nghiệm MapReduce.** Hai cấu hình chạy trên cùng một máy (local-cluster), shuffle giữa executor đi qua loopback nên mạng vật lý không đo được; ba lần lặp chạy lại trên cùng dữ liệu nên khoảng min–max chỉ phản ánh nhiễu của máy; không có Hadoop/HDFS/YARN và không có worker nào bị giết chủ động, nên đường chịu lỗi của cụm chưa được thử.
- **Diễn giải κ.** Các dải Landis–Koch [15] chỉ dùng như quy ước, không phải ngưỡng thống kê; κ được đọc cùng đồng thuận thô vì phụ thuộc tỉ lệ 'yes'.

7.4. Quyền riêng tư và đạo đức

Hệ thống không chứa dữ liệu người thật. learner_id là bút danh HMAC-SHA256 với salt bí mật, không lưu mã gốc; bút danh không phải ẩn danh, người có salt và danh sách mã gốc vẫn nối lại được. Dashboard lớp chỉ đọc số tổng hợp và áp ngưỡng k = 5 (số dưới 5 hiện '<5', còn lại làm tròn tới bội số của 5, ẩn tổng có thể bị suy ngược bằng hiệu), theo tinh thần k-anonymity [10]; làm tròn chỉ cho khoảng, không phải bảo đảm như differential privacy. Kafka, Redis, dịch vụ gợi ý và dashboard chỉ nghe 127.0.0.1; Redis có mật khẩu, Kafka chưa xác thực client. Cảnh báo bế tắc có thể bị dùng để dán nhãn học sinh, nên chỉ hiện ở chế độ giáo viên; lát cắt theo loại kết nối ở mục 6.3 được báo để thấy thiên lệch do xử lý sự kiện muộn.

8. Kết luận và hướng phát triển

Nhóm đã xây dựng một pipeline luồng có trạng thái cho dữ liệu học tập, một dịch vụ gợi ý có kiểm chứng tách khỏi đường luồng, và một bộ đo tái lập được, trong đó mỗi lần chạy để lại một run record và mọi con số của báo cáo được sinh lại từ các bản ghi đó.

- **RQ1:** thời gian phát hiện bế tắc (trung vị) từ 62.947 s (61.267–80.977) với batch theo ngày xuống 3,84 s (3,81–4,03) với streaming (tỉ số 16.388× (15.195–21.237)), đổi bằng một tiến trình Spark chạy liên tục.
- **RQ2:** hint validity 79/120 = 65,8% [57,0–73,7%]; ladder 57/60 = 95,0% [86,3–98,3%] so với template 22/60 = 36,7% [25,6–49,3%]; P95 API 4,9 ms (4,9–5,4) ở 100 người dùng.
- **RQ3:** khớp oracle (trễ high, W = 2 phút) 80,8% (80,2–80,8) với eager và 100,0% (100,0–100,0) với ordered; tỉ lệ sự kiện bị loại ở W = 30 giây: cellular 8,64% (8,58–8,70), wifi 0,85% (0,82–0,86).
- **Hệ thống:** P95 độ trễ cập nhật trạng thái 4,25 s (4,07–4,54) ở 1K ev/s (SLO 5 s); tốc độ bền vững cao nhất quan sát được 10.000 ev/s (lag không tăng nhưng vượt SLO; cao nhất trong SLO: 1.000 ev/s). Nhận định 'đạt SLO 5 s ở 1.000 ev/s với P = 3 hoặc 6' chỉ đúng cho các run 10 phút: chạy 30 phút (soak) thì cấu hình mặc định không giữ được SLO.
- **Chạy dài:** soak 30 phút: P95 độ trễ hệ thống 5,04 s (n = 1) (đạt SLO cả run: 0/1 run), 54,7% (n = 1) số cửa sổ một phút có P95 vượt 5 s, RSS driver tăng 108 MB/phút (n = 1) (5 phút cuối / 5 phút đầu: 1,73× (n = 1)); RSS tăng là rủi ro đã quan sát, nguyên nhân chưa xác định.
- **Số người học:** P95 độ trễ hệ thống 1,78 s (1,76–1,80) ở 100, 4,25 s (4,07–4,54) ở 1.000, 38,08 s (37,92–38,40) ở 10.000 người học; ở 10.000 người học 0/3 run đạt SLO 5 s.
- **Ablation:** khớp oracle 93,1% (93,1–93,9) ở cấu hình mặc định, 21,1% (20,9–21,2) khi tắt khử trùng, 100,0% (100,0–100,0) với ordered (P95 136,85 s (136,53–137,10)) và 93,2% (93,1–93,8) với BKT.
- **Trần ingestion:** 6/6 ô giữ được tốc độ mục tiêu; cao nhất đạt 60.255 ev/s (mục tiêu 60.000 ev/s, 4 tiến trình gửi), nên trần nằm trên mức cao nhất đã thử (60.000 ev/s).
- **MapReduce:** ở 10.000 người học (1.256.768 lần giao), job xong trong 15,10 s (15,04–15,16) với Config A và 8,45 s (8,40–8,58) với Config B (A/B = 1,79× (1,75–1,80)), throughput B 148.730 (146.476–149.633) lần giao/s; 18/18 run khớp chính xác bản tham chiếu tuần tự và số đếm của bộ mô phỏng.

Sản phẩm bàn giao gồm mã nguồn có kiểm thử, ma trận cấu hình máy đọc, kết quả thô chỉ ghi nối, dashboard lớp, và các script sinh hình, báo cáo và slide. Hướng phát triển: chạy trên cụm nhiều nút để đo mở rộng ngang; hiệu chỉnh bộ mô phỏng bằng phân bố tổng hợp từ log thật đã ẩn danh; xác thực theo người học cho API gợi ý; mô phỏng mất mạng theo đợt; và đánh giá tác động học tập của gợi ý với người học thật khi có phê duyệt.

Tài liệu tham khảo

[1] A. T. Corbett, J. R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4(4):253–278, 1995. doi:10.1007/BF01099821

[2] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas, J. Sohl-Dickstein. Deep Knowledge Tracing. arXiv:1506.05908, 2015.

[3] J. E. Beck, Y. Gong. Wheel-spinning: Students who fail to master a skill. AIED 2013, Lecture Notes in Computer Science, pp. 431–440, 2013. doi:10.1007/978-3-642-39112-5_44

[4] N. Matsuda, S. Chandrasekaran, J. Stamper. How quickly can wheel spinning be detected? EDM 2016. https://www.educationaldatamining.org/EDM2016/proceedings/paper_66.pdf

[5] Z. A. Pardos, S. Bhandari. ChatGPT-generated help produces learning gains equivalent to human tutor-authored help on mathematics skills. PLOS ONE, 19(5):e0304013, 2024. doi:10.1371/journal.pone.0304013

[6] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, et al. Structured Streaming: A declarative API for real-time applications in Apache Spark. SIGMOD 2018, pp. 601–613. doi:10.1145/3183713.3190664

[7] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. Fernández-Moctezuma, R. Lax, et al. The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. PVLDB, 8(12):1792–1803, 2015. doi:10.14778/2824032.2824076

[8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, V. Markl. Benchmarking distributed stream data processing systems. ICDE 2018, pp. 1507–1518. doi:10.1109/ICDE.2018.00169

[9] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, et al. SymPy: symbolic computing in Python. PeerJ Computer Science, 3:e103, 2017. doi:10.7717/peerj-cs.103

[10] L. Sweeney. k-anonymity: A model for protecting privacy. Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems, 10(5):557–570, 2002. doi:10.1142/S0218488502001648

[11] Apache Kafka documentation. https://kafka.apache.org/documentation/

[12] Apache Spark. Structured Streaming Programming Guide và trang "Components of a TransformWithState Query" (Spark 4.2.0). https://spark.apache.org/docs/latest/streaming/

[13] PySpark API reference: DataFrame.dropDuplicatesWithinWatermark. https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropDuplicatesWithinWatermark.html

[14] A. Scarlatos, D. Smith, S. Woodhead, A. Lan. Improving the validity of automatically generated feedback via reinforcement learning. Artificial Intelligence in Education (AIED 2024), Lecture Notes in Computer Science, pp. 280–294. Springer Nature Switzerland, 2024. doi:10.1007/978-3-031-64302-6_20 (arXiv:2403.01304)

[15] J. R. Landis, G. G. Koch. The measurement of observer agreement for categorical data. Biometrics, 33(1):159–174, 1977. doi:10.2307/2529310

[16] E. B. Wilson. Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158):209–212, 1927. doi:10.1080/01621459.1927.10502953

Phụ lục A. Khai báo sử dụng AI

Bảng A.1. Nhật ký công cụ, chép từ docs/ai-use-statement.md.

Công cụ	Mục đích
NotebookLM deep research	Tìm nguồn về streaming có trạng thái, knowledge tracing, wheel-spinning, gợi ý có kiểm chứng
Claude Code + skill nature-ref-verifier, nature-statistics, nature-figure (nature-skills commit c1637a8)	Kiểm chứng tài liệu tham khảo; viết giao thức thống kê và figure contract
Claude Code (Claude Opus 5.5)	Sinh mã: JSON Schema, validator, kiểm chứng SymPy, ghi run, script dữ liệu mẫu, test
Claude Code (Claude Opus 5.5)	Implement code cho sản phẩm demo
Claude Code (Claude Opus 5.5)	Viết lời giải từng bước cho kho bài v2 (offline)
Claude Code (Claude Opus 5.5)	Giám sát quá trình chạy ma trận thí nghiệm
Claude Code (Claude Sonnet)	Dịch đề 1.200 bài NuminaMath sang tiếng Việt (statement_vi)

Phụ lục B. Phân công nhiệm vụ

Bảng B.1. Phân công, chép từ contributions.md.

MSSV	Họ tên	Vai trò chính
B25CHHT120	Lê Thiện Văn	Hạ tầng, Kafka ingestion, Spark streaming và trạng thái, bộ chạy thí nghiệm
B25CHHT090	Đinh Hải Dương	Kho bài, bộ kiểm SymPy, bộ mô phỏng, thang gợi ý và dịch vụ gợi ý
B25CHHT121	Nguyễn Quốc Việt	Dữ liệu, quyền riêng tư, batch MapReduce theo thiết kế
B25CHHT116	Đặng Thuỳ Trang	Đánh giá và thống kê (giao thức đo, tóm tắt phiếu chấm)
B25CHHT085	Lê Thiện Đức	Dashboard (quy tắc k = 5)
B25CHHT095	Phạm Minh Hiếu	Người chấm 1 của cả hai phiếu
B25CHHT107	Vũ Hải Nam	Người chấm 2 của cả hai phiếu

Phụ lục C. Trường của một run record

Bảng C.1. Khóa cấp một của run record thuộc ma trận (Phụ lục B của hướng dẫn, thêm runtime_s, spark_conf, docker_images, environment). Mỗi run ghi đúng một dòng vào results/raw/runs.jsonl.

Trường	Nội dung
data_version	Checksum events.parquet của bộ mô phỏng
docker_images	Image Kafka/Redis đang chạy, ghim digest
environment	Model CPU, kernel WSL, giới hạn .wslconfig
experiment	Nhãn thí nghiệm đầu tiên khai báo cấu hình
git	Commit mã nguồn và cờ dirty (chỉ xét src, scripts, configs, contracts, tệp build)
java	Phiên bản JDK dùng cho Spark
log_path	Thư mục artifact của run trong results/raw

machine	CPU, số luồng, RAM, GPU, hệ điều hành
metrics	Độ trễ, throughput, lag, đối soát, tài nguyên, tính hợp lệ
parameters	Toàn bộ tham số cấu hình, groups, seed, topic
run_id	Mã duy nhất: loại run, slug cấu hình, seed, thời điểm
runtime_s	Thời gian chạy thật (đồng hồ tường)
seed	Seed của bộ mô phỏng / kiểm tải
software	Phiên bản Python và các gói ảnh hưởng kết quả
spark_conf	Conf Spark đã dùng (shuffle partitions, state store, jar)
status	success / failed / timeout / aborted
timeout_s	Giới hạn thời gian của run
timestamp	Thời điểm bắt đầu (UTC)

Phụ lục D. Lệnh tái lập

Trên Linux/macOS chạy trực tiếp; trên Windows chạy trong WSL qua scripts/in-wsl.sh. Mọi lệnh đứng ở gốc repo.

```
make up                                    # Kafka + Redis (Docker Compose, cổng
127.0.0.1)
make run-exp EXP=all EXP_ARGS=--dry-run    # ma trận, trạng thái resume, ước lượng giờ
máy
make run-exp EXP=all                       # chạy/tiếp tục ma trận; mỗi run một dòng
runs.jsonl
make review-summary                        # rà soát tay kho bài -> summary.md
make hint-audit-summary                    # audit gợi ý -> docs/hint-audit/summary.md
make mapreduce-matrix                      # MapReduce: quy mô × cấu hình × lần lặp ->
results/mapreduce/
uv run --group report python -m report.figures        # hình H1–H6 -> results/figures/
uv run --group report python -m report.build_report   # báo cáo -> report/bao-cao.docx
uv run --group report python -m slides.build_slides   # slide -> slides/slide.pptx
```

Phụ lục E. Bảng số liệu đầy đủ

Bảng E.1. Exp A: độ trễ và throughput theo tốc độ gửi × số Kafka partition (1K người học, eager). 'Bền vững' = số run có processing lag không tăng / số run. Ô 10.000 ev/s, P = 1: 5 phút cuối chỉ có 1 (1–1) lần đo lag (cần ít nhất 3) vì mỗi micro-batch kéo dài, nên cờ bền vững không xác định; bằng chứng phụ cho thấy không bền vững: tốc độ áp 4.264 (4.256–4.423) ev/s so với tốc độ gửi 10.176 (10.172–10.177) ev/s, lag cuối run 1.809.301 (1.778.904–1.869.016) sự kiện. Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Tốc độ	P	P95 độ trễ hệ thống	P99	Đã áp (ev/s)	Bền vững	Độ dốc lag (ev/s)
100 ev/s	1	1,28 s (1,13–1,63)	1,41 s (1,32–1,75)	99 (99–100)	3/3	0,01 (0,01–0,02)
100 ev/s	3	1,49 s (1,47–1,68)	1,61 s (1,59–1,81)	99 (99–100)	3/3	0,04 (0,04–0,05)
100 ev/s	6	2,22 s (1,75–2,33)	2,41 s (2,03–2,54)	99 (99–100)	3/3	0,05 (0,05–0,07)
1.000 ev/s	1	13,15 s (12,89–13,28)	13,64 s (13,51–14,25)	1.006 (995–1.009)	3/3	1,17 (-0,02–2,13)
1.000 ev/s	3	4,25 s (4,07–4,54)	4,74 s (4,45–4,89)	999 (996–1.003)	3/3	0,04 (-0,07–0,23)
1.000 ev/s	6	3,44 s (3,29–3,83)	3,67 s (3,63–4,02)	1.000 (994–1.000)	3/3	0,19 (-0,06–0,22)
10.000 ev/s	1	506,06 s (499,67–510,25)	527,67 s (521,25–531,79)	4.264 (4.256–4.423)	chưa xác định	không tính được

10.000 ev/s	3	26,78 s (26,75–27,28)	27,85 s (27,69–28,64)	9.926 (9.740–10.005)	3/3	17,54 (-0,19–37,39)
10.000 ev/s	6	11,60 s (11,36–12,03)	11,99 s (11,88–12,43)	9.978 (9.919–10.006)	3/3	-2,09 (-4,38–0,19)

Bảng E.2. Exp B: số người học (kích thước trạng thái) ở 1K ev/s, P = 3. Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Người học	P95 độ trễ hệ thống	Đạt SLO	Lag TB (sự kiện)	Khớp oracle	Recall bế tắc	RSS Spark (TB)	CPU Spark (TB)
100	1,78 s (1,76–1,80)	3/3	1.028 (1.013–1.035)	79,7% (78,5–82,4)	93,2% (89,7–93,2)	3,86 GB (3,73–3,99)	354% (353–354)
1.000	4,25 s (4,07–4,54)	3/3	2.774 (2.731–2.779)	93,1% (93,1–93,9)	97,2% (96,7–97,5)	4,03 GB (3,95–4,06)	422% (420–424)
10.000	38,08 s (37,92–38,40)	0/3	20.681 (20.570–20.867)	97,2% (97,2–97,2)	97,6% (97,4–97,6)	4,67 GB (4,44–4,95)	450% (450–452)

Bảng E.3. Exp D: 14 cấu hình chế độ × watermark × hồ sơ trễ (1K ev/s, P = 3). Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Chế độ	W	Trễ	Khớp oracle	Loại: cellular	Loại: wifi	P95 độ trễ hệ thống
eager	30 s	low	98,4% (98,4–98,6)	2,53% (2,48–2,55)	0,18% (0,18–0,19)	4,82 s (4,48–4,92)
eager	120 s	low	93,1% (93,1–93,9)	0,38% (0,38–0,38)	0,00% (0,00–0,01)	4,25 s (4,07–4,54)
eager	600 s	low	91,8% (91,8–92,1)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	5,05 s (5,01–5,06)
eager	30 s	high	92,4% (92,2–92,8)	8,64% (8,58–8,70)	0,85% (0,82–0,86)	4,89 s (4,84–4,92)
eager	120 s	high	80,8% (80,2–80,8)	3,06% (3,03–3,08)	0,12% (0,12–0,13)	4,99 s (4,98–5,00)
eager	600 s	high	73,8% (73,5–73,9)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	5,06 s (5,00–5,08)
ordered	30 s	low	100,0% (100,0–100,0)	2,48% (2,44–2,49)	0,17% (0,17–0,18)	41,02 s (40,85–41,20)
ordered	120 s	low	100,0% (100,0–100,0)	0,35% (0,34–0,35)	0,00% (0,00–0,00)	136,85 s (136,53–137,10)
ordered	600 s	low	100,0% (100,0–100,0)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	542,00 s (540,35–544,05)
ordered	30 s	high	100,0% (100,0–100,0)	8,56% (8,52–8,63)	0,84% (0,81–0,84)	40,76 s (40,62–40,87)
ordered	120 s	high	100,0% (100,0–100,0)	2,89% (2,88–2,93)	0,11% (0,11–0,12)	136,35 s (136,22–136,77)
ordered	600 s	high	100,0% (100,0–100,0)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	542,28 s (541,67–543,78)
none	—	low	91,6% (91,5–92,1)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	4,66 s (4,63–4,75)
none	—	high	73,7% (73,3–73,8)	0,00% (0,00–0,00)	0,00% (0,00–0,00)	4,71 s (4,61–4,78)

Bảng E.4. Hint validity theo bậc và theo nguồn thực tế của gợi ý.

Tầng	n	Hint validity, cả hai rater [Wilson 95%]

Bậc 1	40	29/40 = 72,5% [57,2–83,9%]
Bậc 2	40	27/40 = 67,5% [52,0–79,9%]
Bậc 3	40	23/40 = 57,5% [42,2–71,5%]
Nguồn: ladder	53	51/53 = 96,2% [87,2–99,0%]
Nguồn: ladder (answer_locked, dạng bước)	7	6/7 = 85,7% [48,7–97,4%]
Nguồn: template	60	22/60 = 36,7% [25,6–49,3%]

Bảng E.5. Trần ingestion của bộ phát lại + Kafka một nút (1 run mỗi ô). 'Lịch gửi' gồm cả bản gửi trùng; 'giữ được tốc độ' = tốc độ đạt ≥ 95% lịch gửi; slip = độ trễ so với lịch gửi của từng sự kiện. Ô ghi trung vị (min–max) qua các seed; 'n = k' khi mới có k run hợp lệ. Nguồn: results/raw/runs.jsonl, các run khớp tiền tố run_id của cấu hình trong plan-v1.0.yaml.

Mục tiêu	Tiến trình gửi	Lịch gửi (ev/s)	Tốc độ đạt (ev/s)	Giữ được tốc độ	Slip P99	CPU Kafka (TB)	CPU bộ phát lại (TB)
30.000 ev/s	1	30.138 (n = 1)	30.136 (n = 1)	1/1	8,3 ms (n = 1)	20% (n = 1)	41% (n = 1)
30.000 ev/s	4	30.138 (n = 1)	30.138 (n = 1)	1/1	8,2 ms (n = 1)	30% (n = 1)	81% (n = 1)
30.000 ev/s	8	30.138 (n = 1)	30.138 (n = 1)	1/1	10,2 ms (n = 1)	40% (n = 1)	113% (n = 1)
60.000 ev/s	1	60.255 (n = 1)	60.172 (n = 1)	1/1	10,2 ms (n = 1)	18% (n = 1)	73% (n = 1)
60.000 ev/s	4	60.255 (n = 1)	60.255 (n = 1)	1/1	9,8 ms (n = 1)	48% (n = 1)	106% (n = 1)
60.000 ev/s	8	60.255 (n = 1)	60.254 (n = 1)	1/1	12,4 ms (n = 1)	39% (n = 1)	171% (n = 1)

Bảng E.6. Ma trận MapReduce: từng lần lặp. Nguồn: results/mapreduce/metrics/<run_key>.json và run record batch-mapreduce cùng run_id trong results/raw/runs.jsonl.

Kịch bản	Lần	run_id	Thời gian job (s)	Throughput (lần giao/s)	CPU executor (%)	Khớp
s1 · 100 · A	1	mr-s1-d1-l100-A-r1-20261028T105929Z	3,207	3.882	201	có
s1 · 100 · A	2	mr-s1-d1-l100-A-r2-20261028T105933Z	3,023	4.118	201	có
s1 · 100 · A	3	mr-s1-d1-l100-A-r3-20261028T105937Z	3,036	4.100	195	có
s2 · 100 · B	1	mr-s2-d1-l100-B-r1-20261028T110056Z	1,793	6.943	413	có
s2 · 100 · B	2	mr-s2-d1-l100-B-r2-20261028T110058Z	1,829	6.806	354	có
s2 · 100 · B	3	mr-s2-d1-l100-B-r3-20261028T110101Z	1,798	6.923	368	có
s3 · 1.000 · A	1	mr-s3-d2-l1000-A-r1-20261028T105940Z	3,985	31.490	194	có
s3 · 1.000 · A	2	mr-s3-d2-l1000-A-r2-20261028T105945Z	3,967	31.632	193	có
s3 · 1.000 · A	3	mr-s3-d2-l1000-A-r3-20261028T105950Z	4,011	31.286	186	có
s4 · 1.000 · B	1	mr-s4-d2-l1000-B-r1-20261028T110104Z	2,254	55.673	369	có
s4 · 1.000 · B	2	mr-s4-d2-l1000-B-r2-20261028T110107Z	2,308	54.370	356	có
s4 · 1.000 · B	3	mr-s4-d2-l1000-B-r3-20261028T110110Z	2,235	56.146	353	có
s5 · 10.000 · A	1	mr-s5-d3-l10000-A-r1-20261028T105955Z	15,039	83.567	208	có
s5 · 10.000 · A	2	mr-s5-d3-l10000-A-r2-20261028T110011Z	15,098	83.241	207	có
s5 · 10.000 · A	3	mr-s5-d3-l10000-A-r3-20261028T110029Z	15,160	82.900	208	có
s6 · 10.000 · B	1	mr-s6-d3-l10000-B-r1-20261028T110113Z	8,399	149.633	410	có

s6 · 10.000 · B	2	mr-s6-d3-l10000-B-r2-20261028T110123Z	8,450	148.730	389	có
s6 · 10.000 · B	3	mr-s6-d3-l10000-B-r3-20261028T110133Z	8,580	146.476	385	có